

Campus Marketplace

Database Schema Design

By Dev-Astators

April 2026

Tech Stack & Architecture

The following schema is designed for the Campus Marketplace project using the agreed stack:

Layer	Technology
Frontend	ReactJS + Vite
Backend	NodeJS + ExpressJS
Database	Supabase (PostgreSQL)
Authentication	Supabase Auth — Google OAuth (university email)
Realtime	Supabase Realtime subscriptions
Storage	Supabase Storage (listing photos)
Payments	PayFast or Peach Payments (TBC)

Authentication Strategy

Students sign in exclusively using their university Google accounts via Supabase Auth (Google OAuth). This means:

- No passwords are stored anywhere in the database
- The profiles table links to auth.users via auth_user_id (UUID)
- Email domain validation should be enforced at the application layer on sign-up
- Role assignment (student, facility_staff, admin) is managed in the profiles table
- Row Level Security (RLS) policies on every table should reference auth.uid()

Entity Relationship Overview

The schema consists of 11 tables grouped into four functional domains:

Domain	Tables	Purpose
Users	profiles	Identity, roles, and reputation
Listings	listings, listing_images, trade_offers	Item catalogue and trade negotiations
Transactions	transactions, facility_bookings, facility_slots, trade_facilities, payments	End-to-end purchase and trade flow
Social	messages, ratings	In-app chat and trust system

Table Schemas

Each table is documented below with column name, data type, constraints, and a description.

User Domain

profiles

Column	Type	Constraint	Description
id	uuid	PK	Primary key, matches auth.users id
auth_user_id	uuid	FK, UNIQUE	References Supabase auth.users(id)
full_name	text	NOT NULL	Student full name from Google OAuth
email	text	UNIQUE	Must be a university Google email
student_number	text	UNIQUE	University student number
university	text	NOT NULL	University name
role	text	NOT NULL	student facility_staff admin
average_rating	numeric	DEFAULT 0	Denormalized avg from ratings table
total_ratings	int	DEFAULT 0	Denormalized count for fast display
created_at	timestamp	DEFAULT now()	Account creation timestamp

Listings Domain

listings

Column	Type	Constraint	Description
id	uuid	PK	Primary key
seller_id	uuid	FK	References profiles(id)
title	text	NOT NULL	Listing title

Column	Type	Constraint	Description
description	text		Full item description
category	text	NOT NULL	e.g. textbooks, electronics, furniture
condition	text	NOT NULL	new like_new good fair poor
asking_price	numeric	NOT NULL	Seller asking price in ZAR
suggested_price	numeric		Derived from SA CPI data integration
listing_type	text	NOT NULL	sale trade both
status	text	DEFAULT active	active reserved sold traded
created_at	timestamp	DEFAULT now()	Listing creation timestamp

listing_images

Column	Type	Constraint	Description
id	uuid	PK	Primary key
listing_id	uuid	FK	References listings(id) ON DELETE CASCADE
storage_path	text	NOT NULL	Supabase Storage bucket path
display_order	int	DEFAULT 0	Order for image carousel display

trade_offers

Column	Type	Constraint	Description
id	uuid	PK	Primary key
listing_id	uuid	FK	The listing being offered on
buyer_id	uuid	FK	References profiles(id)
trade_listing_id	uuid	FK	Buyer's listing offered in exchange
cash_supplement	numeric	DEFAULT 0	Cash top-up offered alongside the trade
status	text	NOT NULL	pending accepted rejected withdrawn
created_at	timestamp	DEFAULT now()	Offer creation timestamp

Transaction Domain

trade_facilities

Column	Type	Constraint	Description
id	uuid	PK	Primary key
name	text	NOT NULL	Facility name
location	text	NOT NULL	Campus building / room
operating_hours	jsonb		{ mon: { open, close }, ... }
slot_capacity	int	NOT NULL	Max bookings per time slot
is_active	boolean	DEFAULT true	Whether facility is currently open

facility_slots

Column	Type	Constraint	Description
id	uuid	PK	Primary key
facility_id	uuid	FK	References trade_facilities(id)
slot_date	date	NOT NULL	Date of this slot
slot_time	time	NOT NULL	Start time of this slot
capacity	int	NOT NULL	Max bookings for this slot
booked_count	int	DEFAULT 0	Current bookings — block slot when = capacity

transactions

Column	Type	Constraint	Description
id	uuid	PK	Primary key
listing_id	uuid	FK	References listings(id)
buyer_id	uuid	FK	References profiles(id)
seller_id	uuid	FK	References profiles(id)
online_amount	numeric	NOT NULL	Amount paid online via payment gateway
cash_shortfall	numeric	DEFAULT 0	Remaining amount to be paid in cash
cash_settled	boolean	DEFAULT false	Staff confirms cash received at facility
status	text	NOT NULL	pending confirmed complete cancelled
created_at	timestamp	DEFAULT now()	Transaction creation timestamp

facility_bookings

Column	Type	Constraint	Description
id	uuid	PK	Primary key
transaction_id	uuid	FK	References transactions(id)
slot_id	uuid	FK	References facility_slots(id)
student_id	uuid	FK	References profiles(id)
booking_type	text	NOT NULL	drop_off collection
status	text	NOT NULL	pending confirmed complete
staff_confirmed_by	text		Staff member who confirmed the handoff
confirmed_at	timestamp		Timestamp of staff confirmation

payments

Column	Type	Constraint	Description
id	uuid	PK	Primary key
transaction_id	uuid	FK	References transactions(id)
gateway_ref	text	UNIQUE	PayFast / Peach Payments reference ID
amount	numeric	NOT NULL	Amount for this payment record in ZAR
payment_type	text	NOT NULL	online cash_supplement
status	text	NOT NULL	pending success failed refunded
paid_at	timestamp		Timestamp of successful payment

Social Domain

messages

Column	Type	Constraint	Description
id	uuid	PK	Primary key
listing_id	uuid	FK	References listings(id) — scopes conversation
sender_id	uuid	FK	References profiles(id)
receiver_id	uuid	FK	References profiles(id)
content	text	NOT NULL	Message body
is_read	boolean	DEFAULT false	Read receipt flag
sent_at	timestamp	DEFAULT now()	Message send timestamp

ratings

Column	Type	Constraint	Description
id	uuid	PK	Primary key
transaction_id	uuid	FK, UNIQUE	One rating pair per transaction
reviewer_id	uuid	FK	References profiles(id)
reviewee_id	uuid	FK	References profiles(id)
score	int	CHECK 1-5	Star rating 1 to 5
review_text	text		Optional written review
is_flagged	boolean	DEFAULT false	Admin moderation flag
created_at	timestamp	DEFAULT now()	Rating submission timestamp

Key Design Decisions

1. No passwords in the database

Supabase Auth with Google OAuth handles all identity. The profiles table stores only a reference to auth.users via auth_user_id. Role and profile data are kept separate from authentication entirely.

2. Denormalised rating fields on profiles

average_rating and total_ratings are stored directly on the profiles table for fast seller profile display, rather than computing them on every query from the ratings table. A database trigger should update these fields whenever a new rating is inserted.

3. Suggested price via SA data integration

The suggested_price field on listings is populated by the backend at listing creation time by querying a South African consumer price data source (Stats SA CPI data by category). This satisfies the SA Data Integration requirement for Project 6.

4. Cash shortfall tracking

The transactions table tracks online_amount and cash_shortfall separately. cash_settled is a boolean toggled by facility staff when they confirm the cash component has been received before releasing items. This flow is reinforced by the facility_bookings table recording which staff member confirmed each handoff.

5. Facility slot capacity management

Trade facilities define a base slot_capacity. Individual facility_slots track booked_count in real time. The backend should block new bookings when booked_count reaches capacity, and should expose available slots to students when booking a drop-off or collection time.

6. Messages scoped to listings

The messages table includes a listing_id foreign key so that each conversation is tied to a specific item. This makes it easy to display the item context alongside the chat, and to clean up messages if a listing is deleted (ON DELETE CASCADE).

7. Supabase Realtime for chat and queue

Supabase Realtime subscriptions on the messages table will power live in-app chat between buyers and sellers. The same mechanism can be used to push queue status updates to students waiting for facility slot confirmations without polling.

8. Row Level Security (RLS)

Every table must have RLS policies enabled in Supabase. Key policies to implement:

- profiles — users can only update their own profile
- listings — only the seller can edit or delete their own listing
- messages — sender and receiver can read; only sender can insert
- transactions — buyer and seller can read their own transactions
- facility_bookings — facility_staff and admins can update status; students read only
- ratings — reviewer can insert once per transaction; both parties can read

Relationship Summary

The key foreign key relationships in the schema are:

From Table	To Table	Relationship
profiles	auth.users	auth_user_id → id (1:1)
listings	profiles	seller_id → id (many:1)
listing_images	listings	listing_id → id (many:1, cascade delete)
trade_offers	listings (x2)	listing_id and trade_listing_id → id
transactions	listings	listing_id → id (many:1)
transactions	profiles (x2)	buyer_id and seller_id → id
facility_bookings	transactions	transaction_id → id (many:1)

From Table	To Table	Relationship
facility_bookings	facility_slots	slot_id → id (many:1)
facility_slots	trade_facilities	facility_id → id (many:1)
payments	transactions	transaction_id → id (many:1)
messages	listings + profiles	listing_id, sender_id, receiver_id
ratings	transactions + profiles	One rating pair per transaction